Νευρωνικά Δίκτυα – Βαθιά Μάθηση: Εργασία 2.

Δημήτριος Μουταφτσίδης 9920 , ΤΗΜΜΥ

23/12/25

1. Abstract

Η παρούσα εργασία αφορά την ταξινόμηση πολλών κλάσεων της βάσης δεδομένων CIFAR-10 μέσω Support Vector Machines (SVM). Ερευνάται η επιρροή των παραμέτρων του μοντέλου στην ακρίβεια και τον χρόνο εκτέλεσης. Συγκρίνεται η επίδοση με τους αλγορίθμους k-NN , NNC και με NN τύπου MLP με Hinge Loss. Περιλαμβάνεται σύντομη θεωρητική ανάλυση των μεθόδων, καθώς και αποτελέσματα πειραμάτων. Θεωρείται συνέχεια των προηγούμενων παραδοτέων. Συνεπώς δεν αναλύονται έννοιες που έχουν ήδη παρουσιαστεί.

2. SVMs.

Πρόκειται για παραμετρικά μοντέλα ταξινόμησης. Τυπικά χρησιμοποιούνται για δυαδικά προβλήματα. Υπάρχουν ,όμως, τεχνικές που μας επιτρέπουν να γενικεύσουμε σε multiclass problems. Η παράμετρος πάνω στην οποία εκπαιδεύουμε είναι το όριο απόφασης μεταξύ των κλάσεων. Κάθε κλάση αντιστοιχίζεται σε μία περιοχή ενός υπερχώρου, με βάση το που είναι τοποθετημένα τα δείγματά της. Νέα datapoints ταξινομούνται σύμφωνα με την θέση τους.

2.1. Γενίκευση σε πολλές κλάσεις.

Χρησιμοποιούνται δύο τεχνικές. **One vs One** και **One vs the Rest.** Για την OvO, δημιουργούνται όλα τα δυνατά ζεύγη κλάσεων. Για κάθε matchup εκπαιδεύεται ένα ξεχωριστό μοντέλο. Συνολικά αντιστοιχεί σε $\frac{n(n-1)}{2}$ διαφορετικά SVMs, όπου n ο αριθμός των κλάσεων. Κατά την άφιξη νέου δείγματος, κάθε μοντέλο ταξινομεί. Η τελική απόφαση λαμβάνεται με βάση ψηφοφορία. Στην περίπτωση των δέκα κλάσεων της CIFAR-10 έχουμε 45 training runs. Η τεχνική είναι απαιτητική σε πόρους. Για την OvtR, αντιθέτως, έχουμε n ακριβώς μοντέλα. Το πρόβλημα δυαδικοποιείται για κάθε κλάση $1,2,\dots,n$ ως 1 vs όλα τα άλλα , 2 vs όλα τα άλλα κ.ο.κ. Για την απόφαση αναφορικά με νέα δείγματα, επιλέγουμε την ταξινόμηση με την μεγαλύτερη βεβαιότητα. Αυτή, δηλαδή, στην οποία το datapoint απέχει περισσότερο από το decision boundary. Είναι αρκετά πιο οικονομική πρακτική. Η παραπάνω διαδικασίες δεν προγραμματίζονται αναλυτικά στην παρούσα εργασία. Τις τρέχει εσωτερικά η Sickit Learn μέσω των εντολών LinearSVC και SVC αντίστοιχα.

2.2. Εκπαίδευση του μοντέλου.

Σκοπός είναι ο καθορισμός ενός hyperplane που δρα ως «σύνορο» μεταξύ τον κλάσεων. Η βασική συνάρτηση απόφασης ορίζεται ως : $f(x) = w^T x + b$, $\hat{y} = sign(f(x))$. Για δείγμα *x,* αποφασίζουμε με βάση το πρόσημο του prediction function. Η διαδικασία εκπαίδευσης συνοψίζεται μαθηματικά ως : $minimize_{w,b,\xi} : \frac{1}{2}||w||^2 + C\sum_{i=1}^{N}\xi_i$. <u>Δηλαδή, βρες τα κατάλληλα βάρη ,slack και bias terms ώστε το hyperplane να απέχει όσο το δυνατόν περισσότερο από τα κοντινότερα δείγματα</u>. Τα ξ είναι slack variables. Επιτρέπουν την λανθασμένη ταξινόμηση εξισορροπώντας επίδοση εκπαίδευσης με την ικανότητα γενίκευσης. Η παράμετρος C ορίζει την «αυστηρότητα» του μοντέλου όσον αφορά τα misclassifications. Η διαδικασία ελαχιστοποίησης εφαρμόζεται υπό τους περιορισμούς $y_i(w^T x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$. Η παραπάνω μέθοδος λειτουργεί καλύτερα για γραμμικά διαχωρίσιμα δεδομένα. Κάτι τέτοιο σπάνια συμβαίνει. Τα SVM ξεπερνούν αυτό το πρόβλημα μέσω της χρήσης Kernel Functions. Προβάλουν τα δεδομένα σε μεγαλύτερη διάσταση, όπου ο γραμμικός διαχωρισμός είναι πλέον εφικτός. Η συνάρτηση απόφασης ορίζεται πλέον ως $f(x) = \sum_{i \in SV} a_i y_i K(x_i, x) + b$. Μετά το πέρας της διαδικασίας εκπαίδευσης, η τοποθέτηση του ορίου στον υπερχώρο καταλήγει να εξαρτάται από ορισμένα μόνο features vectors. Τα πλησιέστερα σε αυτό. Τα διανύσματα αυτά καλούνται **Support Vectors** και είναι η κινητήρια δύναμη πίσω από την ταξινόμηση.

Είναι τα μόνα διανύσματα που συμμετέχουν στην παραπάνω σχέση. Εξ 'ου και $i \in SV$.

2.3. Τύποι SVM.

Στην παρούσα εργασία έγινε χρήση τριών τύπων SVM με βάση την συνάρτηση πυρήνα τους. Linear, Polynomial και Gaussian.

2.3.1. Linear Kernel.

Απλό και οικονομικό σε επεξεργαστική ισχύ. Χρησιμοποιεί by default OvtR approach στην Sickit Learn (module LinearSVC). Δεν αποδείχθηκε πολύ ικανό για τα συγκεκριμένα δεδομένα. Η προς διερεύνηση παράμετρος στην συγκεκριμένη περίπτωση είναι η **C**. Είναι η «αυστηρότητα» όσον αφορά τα misclassifications που είδαμε προηγουμένως. Μεγαλύτερο C → πιο περίπλοκα όρια, αλλά με ρίσκο το overfitting.

2.3.2. Polynomial Kernel.

Ο πυρήνας ορίζεται ως $K(y,z) = (\gamma y^T z + coef)^d$. Οι προς διερεύνηση παράμετροι είναι το **C**, ο βαθμός ***d*** του πολυωνύμου, το scaling factor $\boldsymbol{\gamma}$ και η σταθερά ***coef***. Πολυώνυμα μεγαλύτερου βαθμού τείνουν να μπορούν να διαχειριστούν πιο περίπλοκα όρια απόφασης, όμως είναι πιο επιρρεπή σε overfitting. Μεγαλύτερο γ αυξάνει την ευαισθησία του πυρήνα στα δεδομένα. Έτσι να όρια γίνονται πιο «ζωηρά». Το tradeoff είναι ίδιο με πριν. Τέλος, η σταθερά επηρεάζει την γενικότερη συμπεριφορά του μοντέλου. Τα SVMs με Polynomial Kernel είναι αρκετά απαιτητικά σε πόρους. Χρησιμοποιούν by default OvO approach.

2.3.3. RBF – Gaussian Kernel.

Ο πυρήνας ορίζεται ως $K(x,z) = \exp\left(-\gamma \left|\left|x - z\right|\right|^2\right)$. Οι παράμετροι που εξετάζουμε είναι οι **C** και **γ** που αναφέρθηκαν προηγουμένως. Είναι πολύ απαιτητικό σε επεξεργαστική ισχύ. Χρησιμοποιεί by default OvO.

2.4. Class Weights.

Χρησιμοποιούνται σε προβλήματα στα οποία κάποιες κλάσεις είναι πιο πιθανές από άλλες. Κάνουν τα μοντέλα να είναι λιγότερα biased ως προς τις συχνότερες κλάσεις, αντισταθμίζοντας έτσι την ανισορροπία. Στην περίπτωσή μας δεν χρησιμοποιούνται. Η CIFAR-10 είναι ομοιόμορφη.

2.5. Standard Scaler.

Πρόκειται για κανονικοποίηση των δεδομένων ώστε να έχουν **μηδενική μέση τιμή και μοναδιαία τυπική απόκλιση.** Εφαρμόζει τον τύπο: $\frac{x-\mu}{\sigma}$. Είναι σχεδόν απαραίτητη μετατροπή. Ειδικά για RBF και Polynomial kernels. Τοποθετεί όλα τα features στην ίδια κλίμακα. Χωρίς Scaling, διανύσματα μεγάλου αριθμητικού εύρους κυριαρχούν έναντι άλλων. Τα dot products των kernels χάνουν κάπως το νόημά τους. Προσοχή: <u>ο υπολογισμός μέσης τιμής και τυπικής απόκλισης γίνεται με βάση τα στατιστικά των training data και εφαρμόζεται με τον ίδιο τρόπο στο test set. Σε αντίθετη περίπτωση έχουμε information leak</u>.

3. SVMs – Πειράματα.

Για την κατάλληλη επιλογή kernel type και παραμέτρων, έγιναν πειράματα τύπου Grid Search. Χρησιμοποιήθηκαν τρεις προσεγγίσεις , για 6 συνολικά πειράματα. Χρήση των έγχρωμων pixels, χρήση ασπρόμαυρων εικόνων με και χωρίς PCA, χρήση του μέτρου του Μετασχηματισμού Fourier των εικόνων ως input με και χωρίς PCA. Τέλος, το κυρίαρχο μοντέλο απομονώθηκε για ένα πιο αναλυτικό Grid Search. Όπου έγινε PCA, διατηρήθηκε το 90% της πληροφορίας σύμφωνα με την υπόδειξη της εκφώνησης. Στο στάδιο του πειραματισμού για επιλογή παραμέτρων χρησιμοποιήθηκε k-fold validation έναντι της αξιολόγησης με βάση το test set. Αυτό ήταν ένα (περίπου) λάθος που έγινε στην προηγούμενη εργασία με το CNN. Πλέον τα δύο σετ είναι πλήρως απομονωμένα. Αποφεύγεται έτσι leak πληροφορίας. Τα αναλυτικά αποτελέσματα των Grid Search, όπως αυτά παράγονται από την SkLearn, αποθηκεύτηκαν σε αρχεία Microsoft Excel υπό την μορφή spreadsheet για εύκολη και φιλική προς τον χρήστη (εμένα) αξιολόγηση. Τα αρχεία αυτά επισυνάπτονται. Για οικονομία χρόνου και υπολογιστικών πόρων, η φάση του πειραματισμού έγινε σε stratified υποσύνολο των εικόνων. Μετά το πέρας των πειραμάτων επιλογής μοντέλου, ο «νικητής» εκπαιδεύτηκε σε ολόκληρο το training set και αξιολογήθηκε με βάση τα test data για την τελική επίδοση.

3.1. Πείραμα 1 – RGB εικόνες.

Setup:

Αρχείο EXCEL	Plain_SVM.xlsx
Folds	3
Samples	10.000
PCA	90%
Προεπεξεργασία	---

Βέλτιστο Μοντέλο:

Kernel	RBF
C	5.0
Gamma	Scale
Mean Test Score	46.65%
Mean Train Score	92.22%
Χρόνος Εκπαίδευσης	60 sec.
Χρόνος Ταξινόμησης	4.7 sec.

Σχόλια:

1. RBF και Polynomial Kernels είχαν σχεδόν ισοδύναμες βέλτιστες επιδόσεις.

2. Η επιλογή των παραμέτρων στα RBF και Poly είναι ύψιστης σημασίας. Παρατηρούμε πολύ μεγάλη διαφορά στην επίδοση μεταξύ των διαφορετικών setups. Το συγκεκριμένο δίνει επίδοση 46%. Για C = 1 , Gamma = 0.001 πέφτει στο 31%.

3. Τα Linear SVMs αποδίδουν χαμηλά , αλλά σταθερά. Η επίδοση τους είναι σταθερά 37%. Η μεταβολή του C δεν τα επηρεάζει σχεδόν καθόλου.

4. Το Mean Training Score δεν συσχετίζεται απαραίτητα με το Mean Test Score. Έχουμε μοντέλο με επίδοση εκπαίδευσης 99% και επίδοση 31% (Row 33 στο αρχείο). Έχουμε και μοντέλο με 62% και 44% αντίστοιχα (Row 9).

3.2. Πείραμα 2 – Grayscale Εικόνες.

Setup:

Αρχείο EXCEL	Grayscale_SVM_No_PCA.xlsx
Folds	3
Samples	10.000
PCA	---
Προεπεξεργασία	Grayscale()

Βέλτιστο Μοντέλο:

Kernel	RBF
C	5.0
Gamma	Scale
Mean Test Score	38.34%
Mean Train Score	94,8%
Χρόνος Εκπαίδευσης	182 sec.
Χρόνος Ταξινόμησης	43 sec.

Σχόλια:

1. Η επίδοση μειώθηκε αισθητά. Ίσως αναμενόμενη συμπεριφορά, αφού η απώλεια χρώματος συνεπάγεται και απώλεια πληροφορίας. Θα επιβεβαιώσουμε αργότερα πως δεν ευθύνεται η απουσία PCA.

2. Οι χρόνοι εκπαίδευσης και ταξινόμησης αυξήθηκαν κατά πολύ. Στο επόμενο πείραμα θα επιβεβαιωθεί πως ευθύνεται η απουσία PCA.

3. Τα Linear μοντέλα πλέον αποδίδουν πολύ χαμηλά. Είναι τελευταία στην κατάταξη με επίδοση μόλις 19%.

3.3. Πείραμα 3 - Grayscale εικόνες.

Setup:

Αρχείο EXCEL	Grayscale_SVM.xlsx
Folds	3
Samples	10.000
PCA	90%
Προεπεξεργασία	Grayscale

Βέλτιστο Μοντέλο:

Kernel	RBF
C	5.0
Gamma	Scale
Mean Test Score	38.48%
Mean Train Score	86.5%
Χρόνος Εκπαίδευσης	13 sec.
Χρόνος Ταξινόμησης	4.6 sec.

Σχόλια:

1. Το PCA βελτίωσε κατά πολύ τον χρόνο εκτέλεσης. Όχι όμως την επίδοση. Το μοντέλο είναι αισθητά πιο γρήγορο και από αυτό του Πειράματος 1. Αναμενόμενο, αφού τα feature vectors είναι μικρότερα.

2. Το RBF Kernel με τις ίδιες παραμέτρους παραμένει σταθερά το καλύτερο.

3. Μετά το PCA, η επίδοση των Linear SVMs αυξήθηκε κατά περίπου 10%. Επωφελούνται από το Dimensionality Reduction.

3.4. Πείραμα 4 – Fourier Transform.

Setup:

Αρχείο EXCEL	FT_SVM_No_PCA.xlsx
Folds	3
Samples	10.000
PCA	---
Προεπεξεργασία	Grayscale() , Frequency_Content()

Βέλτιστο Μοντέλο:

Kernel	RBF
C	1.0
Gamma	Scale
Mean Test Score	39.74%
Mean Train Score	82.33%
Χρόνος Εκπαίδευσης	207 sec.
Χρόνος Ταξινόμησης	63 sec.

Σχόλια:

1. Ελαφρώς καλύτερη επίδοση από την Grayscale εκδοχή. Χωρίς αξιοσημείωτες διαφορές.

2. Φαίνεται να ευνοεί μικρότερες τιμές του C. Ορατό σε όλη την πρώτη πεντάδα.

3. Είναι το πιο αργό μοντέλο μέχρι τώρα.

3.5. Πείραμα 5 – Fourier Transform.

Setup:

Αρχείο EXCEL	FT_SVM.xlsx
Folds	3
Samples	10.000
PCA	90%
Προεπεξεργασία	Grayscale() , Frequency_Content()

Βέλτιστο Μοντέλο:

Kernel	RBF
C	1.0
Gamma	0.001
Mean Test Score	39.96%
Mean Train Score	77.36%
Χρόνος Εκπαίδευσης	92 sec.
Χρόνος Ταξινόμησης	23 sec.

Σχόλια:

1. Το μοντέλο παραμένει αργό. Ωστόσο το PCA βοήθησε αρκετά. Δεν επηρέασε την επίδοση.

3.6. Γενικότερα σχόλια για τα Πειράματα 1-5.

Τα μοντέλα με RBF Kernel είναι τα καλύτερα. Επίσης αξιοσημείωτα είναι τα Πολυωνυμικά. Τα Γραμμικά είναι πολύ πιο γρήγορα, ωστόσο υστερούν σε ακρίβεια.

Η βέλτιστη μορφή ταξινόμησης των εικόνων είναι η RGB. Οι άλλες δύο φαίνεται να στερούν πληροφορία.

Προσωπικό Σημείωμα: Για τον FT συγκεκριμένα, δεν έχω κάποια τεκμηρίωση όσον αφορά το γιατί επιλέχθηκε. Είχα απλώς περιέργεια για το πως θα επηρεάσει τα αποτελέσματα. Δεν ήξερα τι να περιμένω.

Τώρα που κατασταλάξαμε σε αρχιτεκτονική μοντέλου, θα προχωρήσουμε σε πιο αναλυτικό Grid Search, σε μεγαλύτερο υποσύνολο των εικόνων.

3.7. Πείραμα 6 – RBF-specific Grid Search.

Setup:

Αρχείο EXCEL	Plain_SVM_RBF.xlsx
Folds	5
Samples	20.000
PCA	90%
Προεπεξεργασία	---

Βέλτιστο Μοντέλο:

Kernel	RBF
C	5.0/4.0
Gamma	Scale
Mean Test Score	50.06%
Mean Train Score	89.98%

Χρόνος Εκπαίδευσης	230 sec.
Χρόνος Ταξινόμησης	19.58 sec.

Σχόλια:

1. Οι μεγαλύτεροι χρόνοι προκύπτουν λόγω των περισσότερων δειγμάτων.

2. C = 4 και C = 5 δίνουν ισοδύναμα αποτελέσματα.

3. Το μοντέλο είναι πιο ευαίσθητο στην επιλογή του Gamma.

4. Η επίδοση είναι αρκετά σταθερή. Το καλύτερο με το χειρότερο run διαφέρουν κατά 2%.

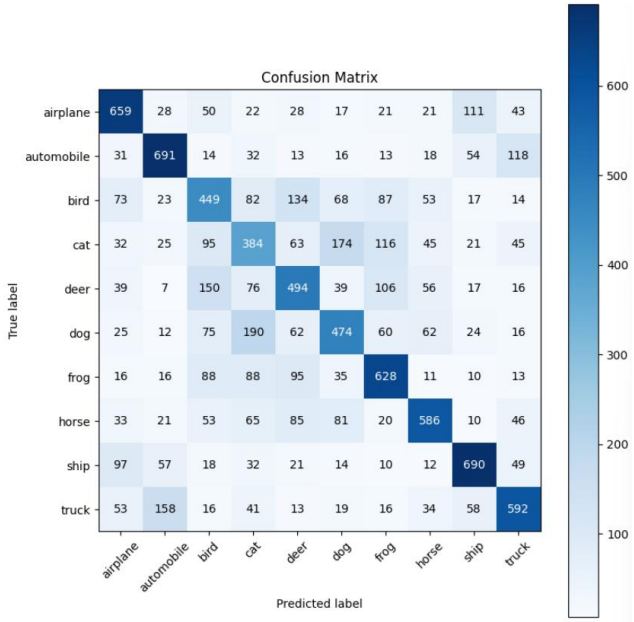
3.8. Τελικό Train και Test Run.

Πλέον εκπαιδεύουμε σε όλο το dataset με C = 5.0 και Gamma = Scale. Αρχικά έχουμε αποτελέσματα **με PCA.**

```
Train time (s): 358.8365740776062
Test time (s): 58.97232365608215
Test accuracy: 0.5647
```

Classification report:

	precision	recall	f1-score	support
0	0.62	0.66	0.64	1000
1	0.67	0.69	0.68	1000
2	0.45	0.45	0.45	1000
3	0.38	0.38	0.38	1000
4	0.49	0.49	0.49	1000
5	0.51	0.47	0.49	1000
6	0.58	0.63	0.60	1000
7	0.65	0.59	0.62	1000
8	0.68	0.69	0.69	1000
9	0.62	0.59	0.61	1000
accuracy			0.56	10000
macro avg	0.56	0.56	0.56	10000
weighted avg	0.56	0.56	0.56	10000

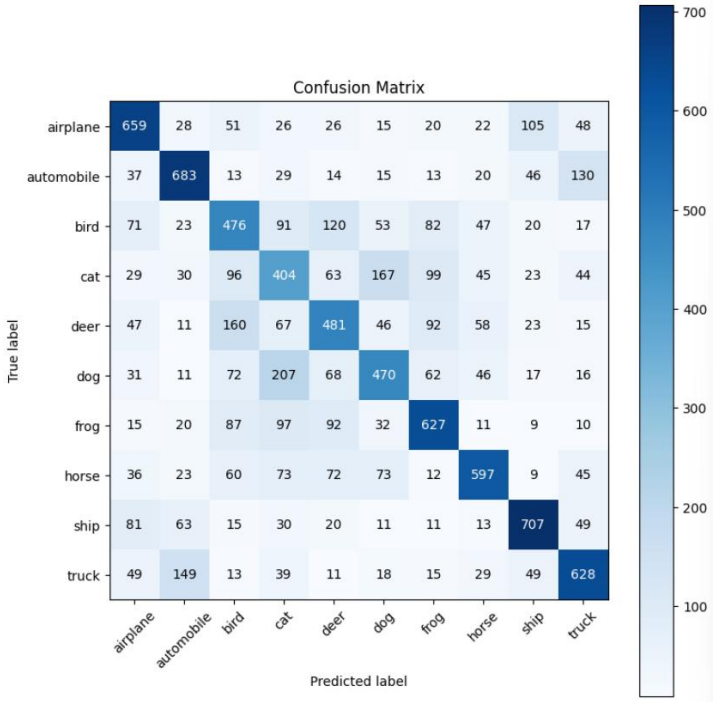


Η επίδοση είναι καλύτερη. Αναμενόμενο, αφού πλέον αξιοποιούμε και τις 50.000 εικόνες. Ο χρόνος είναι σαφώς μεγαλύτερος. Θυμίζουμε πως αυτή είναι η πρώτη αντικειμενική αξιολόγηση του μοντέλου σε άγνωστα δεδομένα. Ακολουθούν τα αποτελέσματα **χωρίς την χρήση PCA.**

```
Train time (s): 5783.787525892258
Test time (s): 1368.8668150901794
Test accuracy: 0.5732
```

Classification report:

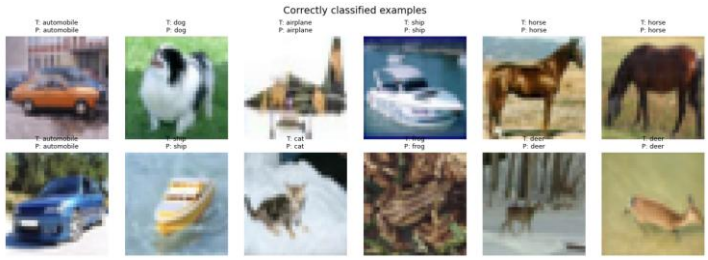
	precision	recall	f1-score	support
0	0.62	0.66	0.64	1000
1	0.66	0.68	0.67	1000
2	0.46	0.48	0.47	1000
3	0.38	0.40	0.39	1000
4	0.50	0.48	0.49	1000
5	0.52	0.47	0.49	1000
6	0.61	0.63	0.62	1000
7	0.67	0.60	0.63	1000
8	0.70	0.71	0.70	1000
9	0.63	0.63	0.63	1000
accuracy			0.57	10000
macro avg	0.57	0.57	0.57	10000
weighted avg	0.57	0.57	0.57	10000



Συμπεραίνουμε τα εξής για το PCA: δεν επηρεάζει ουσιαστικά την επίδοση του μοντέλου. Η κατανομή της ακρίβειας ανά κλάση παραμένει ίδια. Αυτό που αλλάζει ριζικά είναι ο χρόνος εκπαίδευσης. Το μοντέλο είναι ανυπόφορα αργό χωρίς PCA.

3.9. Ορθή και λάθος ταξινόμηση.

Ακολουθούν ενδεικτικά παραδείγματα ορθής και λανθασμένης ταξινόμησης.



4. MLP – Hinge Loss.

4.1. MLP.

Ο Multilayer Perceptron πρόκειται για μία διαδομένη και απλή μορφή Νευρωνικού Δικτύου. Διαφέρει από το CNN της πρώτης εργασίας κυρίως στο ότι δέχεται flat input. Δεν είναι τόσο ικανό στα image classification tasks, καθώς δεν λαμβάνει υπόψιν την χωρική δομή των εικόνων. Ειδικά με ένα μόλις Hidden Layer, το learning capacity είναι περιορισμένο. Σημειώνουμε πως για της ανάγκης αυτής της εργασίας, το Batch Normalization δεν προσμετράται ως layer. Επίσης έννοιες όπως Optimizer , Scheduler , Data Augmentations θεωρούνται γνωστές από την πρώτη εργασία. Η διαδικασία εκπαίδευσης συνοψίζεται μαθηματικά ως εξής:

1. Pre-activation: $z = W_1 x + b_1$.

2. Activation: $a = \varphi(z)$, όπου $\varphi \rightarrow ReLU$.

3. Scoring: $s = W_2 a + b_2$.

4. Prediction: $\hat{y} = argmax\{s_k\}, k\epsilon\{1, \dots, K\}$.

Η εκπαίδευση αποσκοπεί στον ορθό προσδιορισμό των πινάκων βαρών W και Bias b.

4.2. Hinge Loss.

Πρόκειται για την φιλοσοφία του SVM. Μεγιστοποιεί margins μεταξύ κλάσεων. Φροντίζει η ορθή κλάση να έχει Score που ξεπερνά αυτό των υπολοίπων για κάποιο όριο. Δουλεύουμε δηλαδή με scores, όχι πιθανότητες όπως στην περίπτωση του Cross Entropy Loss. Για ένα δείγμα που ανήκει στην κλάση y, το HL ορίζεται ως: $L(s, y) = \sum_{j \neq y} \max(0, 1 + s_j - s_y)$, όπου s το διάνυσμα των scores.

4.3. Επίδοση MLP – Πείραμα.

Παρουσιάζονται τα αποτελέσματα για το εξής Setup:

Data Augmentation: Random Crop , Flip , Color Jitter , Random Erasing.

Scheduler: Step (20 Epochs)

Optimizer: Adam

Batch Normalization: Ναι

Batch Size: 128

Dropout: 0.2

Weight Decay: 10^{-3}

LR: 10^{-3}

Epochs: 60

Activation Function: ReLU

Φαίνονται επακριβώς στο configuration dictionary του κώδικα.

Έγιναν διάφορα γρήγορα πειράματα. Παρουσιάζεται το πιο ικανοποιητικό μοντέλο. Η χρήση Data Augmentation είναι λίγο «άδικη» για το SVM. Έγιναν πρόχειρες προσπάθειες ένταξης αυτής της διαδικασίας και στο SVM, δίχως όμως αποτέλεσμα. Έτσι θεωρήθηκε καλό να συγκριθούν τα δύο μοντέλα στο peak της ισχύος τους.

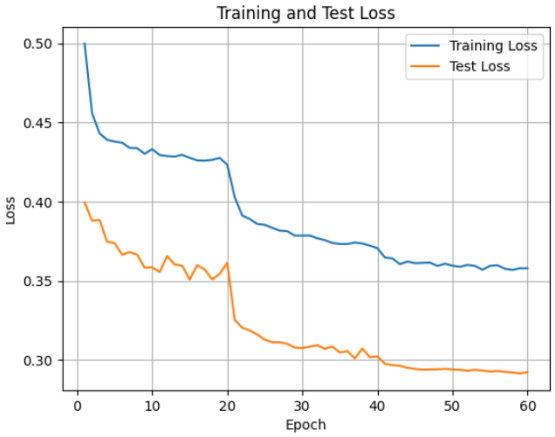
Σημείωμα: Στην προηγούμενη εργασία έγινε το εξής λάθος: εκπαιδεύαμε στο Training Set και αξιολογούσαμε κάθε epoch πάνω στο Test Set. Η διαδικασία αυτή διαρρέει πληροφορία. Παρόλο που το μοντέλο δεν βλέπει ποτέ τα Test Data, τα βλέπουμε «εμείς οι ερευνητές». Μπορεί λοιπόν κάποιος να ισχυριστεί ότι εξ' ορισμού το μοντέλο μας είναι σχεδιασμένο να αποδίδει καλά στο συγκεκριμένο test set. Κατόπιν συζήτησης με τον διδάσκοντα ωστόσο, συμπεραίνουμε το εξής: η «αμαρτία» αυτή είναι επιτρεπτή , αρκεί να παράξουμε ένα μοντέλο που σταθερά και για πολλά epochs κυμαίνεται γύρω από μία σταθερή επίδοση. Έτσι ξέρουμε πως η επιτυχία του μοντέλου δεν ήταν one-time-event. Σε αυτό το πείραμα ξαναχρησιμοποιούμε την παραπάνω φιλοσοφία, αλλά πλέον με γνώση.

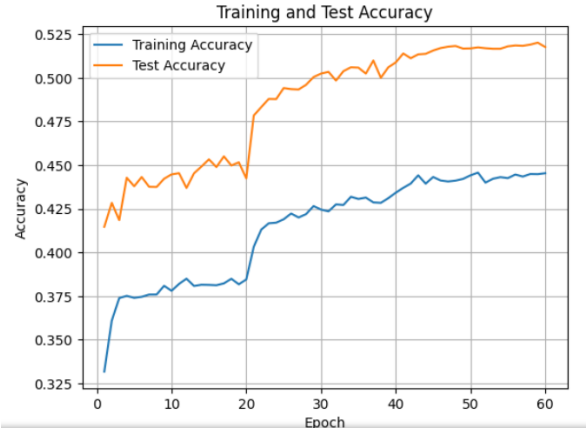
Αποτελέσματα:

Μέγιστη Ακρίβεια: 52%

Τυπική Ακρίβεια: Περίπου 51%

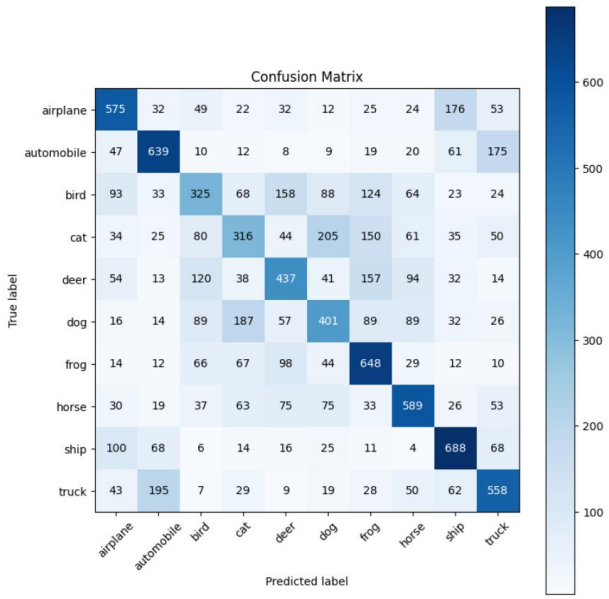
Χρόνος Εκτέλεσης: 62 λεπτά (περίπου ένα λεπτό ανά epoch)





Classification Report:

	precision	recall	f1-score	support
airplane	0.57	0.57	0.57	1000
automobile	0.61	0.64	0.62	1000
bird	0.41	0.33	0.36	1000
cat	0.39	0.32	0.35	1000
deer	0.47	0.44	0.45	1000
dog	0.44	0.40	0.42	1000
frog	0.50	0.65	0.57	1000
horse	0.58	0.59	0.58	1000
ship	0.60	0.69	0.64	1000
truck	0.54	0.56	0.55	1000
accuracy			0.52	10000
macro avg	0.51	0.52	0.51	10000
weighted avg	0.51	0.52	0.51	10000



Σχόλια:

Βλέπουμε πως για τα δέκα τελευταία epochs το μοντέλο αποδίδει σταθερά γύρω στο 51%. Επομένως μπορούμε να πούμε πως τα αποτελέσματα είναι καλά. Το δίκτυο είναι αρκετά πιο γρήγορο σε σχέση με το CNN, αλλά και λιγότερο ικανό.

5. Σύγκριση SVM – k-NN – NCC – MLP.

Το SVM αποδίδει σαφώς καλύτερα σε σχέση με τους απλούστερους k-NN και NCC. Η αρχιτεκτονική του είναι ικανή να διαχειριστεί πιο περίπλοκους κανόνες αποφάσεων. Έχει επομένως μεγαλύτερο learning capacity. Είναι , ωστόσο, πιο αργό.

Το SVM υπερτερεί έναντι και του MLP. Η επίδοση είναι μόλις λίγο καλύτερη, ωστόσο πρέπει να αναλογιστούμε την πολυπλοκότητα και την βραδύτητα εκπαίδευσης του δεύτερου. Έχουμε πολύ περισσότερους παράγοντες να μελετήσουμε και η εκπαίδευση είναι πολύ πιο αργή. Grid Search-like προσεγγίσεις για tuning είναι πολύ δύσκολο να υλοποιηθούν.

6. Προσωπικό Σημείωμα.

Τι έμαθα:

Νιώθω ότι «σκόρπισα» τον κώδικα λιγότερο σε σχέση με την Εργασία 1. Σταμάτησα να προσπαθώ να κάνω τα πάντα Modular και γραμμένα σε Custom συναρτήσεις. Επίσης αξιοποίησα περισσότερο built-in μεθόδους. Έτσι κέρδισα σε δύο τομείς. Ταλαιπωρήθηκα πολύ λιγότερο στο γράψιμο. Δεν χρειάστηκε πολύ debugging και ο κώδικας ήταν πολύ πιο σύντομος και τακτοποιημένος. Επίσης νομίζω πως έτρεχε και γρηγορότερα. Ακόμα, μου άρεσε σαν διευκόλυνση η αποθήκευση των αποτελεσμάτων σε Excel. Μπορούσα να τα ελέγχω χαλαρά και αναλυτικά σε πολύ φιλικό περιβάλλον. Τέλος, ξεκαθάρισα την διαφορά CV και Information Leakage που μου είχε ξεφύγει στην προηγούμενη εργασία.